```
CREATE TABLE ER_DEPARTMENT (
   DEPT_ID NUMBER PRIMARY KEY,
   DEPT_NAME VARCHAR2(50) UNIQUE NOT NULL
);

CREATE TABLE ER_JOB (
   JOB_ID NUMBER PRIMARY KEY,
   JOB_TITLE VARCHAR2(100) NOT NULL,
   DEPT_ID NUMBER NOT NULL,
   JOB_DESCRIPTION VARCHAR2(500),
   MIN_SALARY NUMBER(10,2),
   MAX_SALARY NUMBER(10,2),
   VACANCIES NUMBER DEFAULT 1 NOT NULL,
   JOB_STATUS VARCHAR2(20) DEFAULT 'OPEN',
   POSTED_DATE DATE DEFAULT SYSDATE,
   CONSTRAINT ER_JOB_DEPT_FK
      FOREIGN KEY (DEPT_ID)
      REFERENCES ER_DEPARTMENT(DEPT_ID),
   CONSTRAINT ER_JOB_SALARY_CK
      CHECK (MIN_SALARY IS NULL OR MAX_SALARY IS NULL
           OR MIN_SALARY <= MAX_SALARY),
   CONSTRAINT ER_JOB_VACANCY_CK CHECK (VACANCIES >= 0),
   CONSTRAINT ER_JOB_STATUS_CK
      CHECK (JOB_STATUS IN ('OPEN', 'CLOSED', 'ON HOLD'))
);
```

```
CREATE TABLE ER_CANDIDATE (
   CANDIDATE_ID NUMBER PRIMARY KEY,
   FULL_NAME VARCHAR2(100) NOT NULL,
   EMAIL VARCHAR2(100) UNIQUE NOT NULL,
   PHONE VARCHAR2(15),
   QUALIFICATION VARCHAR2(100),
   EXPERIENCE_YEARS NUMBER(3,1) DEFAULT 0,
   CREATED_DATE DATE DEFAULT SYSDATE,
   CONSTRAINT ER_CAND_EXP_CK CHECK (EXPERIENCE_YEARS >= 0)
);

CREATE TABLE ER_APPLICATION (
   APPLICATION_ID NUMBER PRIMARY KEY,
   CANDIDATE_ID NUMBER NOT NULL,
   JOB_ID NUMBER NOT NULL,
   APPLICATION_DATE DATE DEFAULT SYSDATE,
   APPLICATION_STATUS VARCHAR2(25) DEFAULT 'APPLIED',
   REMARKS VARCHAR2(500),
   CONSTRAINT ER_APP_CAND_FK
      FOREIGN KEY (CANDIDATE_ID)
      REFERENCES ER_CANDIDATE(CANDIDATE_ID),
   CONSTRAINT ER_APP_JOB_FK
      FOREIGN KEY (JOB_ID)
      REFERENCES ER_JOB(JOB_ID),
   CONSTRAINT ER_APP_UNQ UNIQUE (CANDIDATE_ID, JOB_ID),
```

```
  CONSTRAINT ER_APP_STATUS_CK
    CHECK (APPLICATION_STATUS IN (
      'APPLIED', 'SHORTLISTED', 'INTERVIEW',
      'SELECTED', 'REJECTED', 'HIRED'
    ))
);




CREATE TABLE ER_INTERVIEW (
  INTERVIEW_ID NUMBER PRIMARY KEY,
  APPLICATION_ID NUMBER NOT NULL,
  INTERVIEW_ROUND NUMBER DEFAULT 1,
  INTERVIEW_DATE DATE NOT NULL,
  INTERVIEW_MODE VARCHAR2(20),
  INTERVIEWER_NAME VARCHAR2(100),
  SCORE NUMBER(5,2),
  FEEDBACK VARCHAR2(500),
  INTERVIEW_STATUS VARCHAR2(20) DEFAULT 'SCHEDULED',
  CONSTRAINT ER_INT_APP_FK
    FOREIGN KEY (APPLICATION_ID)
    REFERENCES ER_APPLICATION(APPLICATION_ID),
  CONSTRAINT ER_INT_UNQ
    UNIQUE (APPLICATION_ID, INTERVIEW_ROUND),
  CONSTRAINT ER_INT_SCORE_CK
    CHECK (SCORE IS NULL OR SCORE BETWEEN 0 AND 100),
  CONSTRAINT ER_INT_MODE_CK
```

```
    CHECK (INTERVIEW_MODE IN ('ONLINE', 'OFFLINE')),
  CONSTRAINT ER_INT_STATUS_CK
    CHECK (INTERVIEW_STATUS IN (
      'SCHEDULED', 'COMPLETED', 'CANCELLED'
    ))
);

CREATE TABLE ER_EMPLOYEE (
  EMPLOYEE_ID NUMBER PRIMARY KEY,
  APPLICATION_ID NUMBER UNIQUE NOT NULL,
  CANDIDATE_ID NUMBER UNIQUE NOT NULL,
  JOINING_DATE DATE,
  DESIGNATION VARCHAR2(100),
  ANNUAL_SALARY NUMBER(12,2),
  EMPLOYMENT_STATUS VARCHAR2(20) DEFAULT 'ACTIVE',
  CONSTRAINT ER_EMP_APP_FK
    FOREIGN KEY (APPLICATION_ID)
    REFERENCES ER_APPLICATION(APPLICATION_ID),
  CONSTRAINT ER_EMP_CAND_FK
    FOREIGN KEY (CANDIDATE_ID)
    REFERENCES ER_CANDIDATE(CANDIDATE_ID),
  CONSTRAINT ER_EMP_STATUS_CK
    CHECK (EMPLOYMENT_STATUS IN ('ACTIVE', 'INACTIVE'))
);
```

```
INSERT INTO ER_DEPARTMENT VALUES (1, 'Human Resources');
INSERT INTO ER_DEPARTMENT VALUES (2, 'Information Technology');
INSERT INTO ER_DEPARTMENT VALUES (3, 'Finance');
INSERT INTO ER_DEPARTMENT VALUES (4, 'Data Science');
INSERT INTO ER_DEPARTMENT VALUES (5, 'DBMS and Database Administration');
INSERT INTO ER_DEPARTMENT VALUES (6, 'Software Development');
INSERT INTO ER_DEPARTMENT VALUES (7, 'Data Analytics');

COMMIT;
```

```
INSERT INTO ER_JOB
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(101, 'HR Executive', 1, 'Recruitment and employee coordination',
 300000, 500000, 3);

INSERT INTO ER_JOB
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(102, 'Software Developer', 2, 'Software development and support',
 450000, 900000, 4);

INSERT INTO ER_JOB
```

```
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(103, 'Financial Analyst', 3, 'Financial planning and reporting',
 350000, 650000, 2);

INSERT INTO ER_JOB
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(104, 'Data Scientist', 4, 'Data modelling and machine learning',
 500000, 1000000, 3);

INSERT INTO ER_JOB
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(105, 'Database Administrator', 5, 'Database management and SQL',
 400000, 800000, 2);

INSERT INTO ER_JOB
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(106, 'Application Developer', 6, 'Design and develop applications',
 400000, 850000, 3);

INSERT INTO ER_JOB
```

```
(JOB_ID, JOB_TITLE, DEPT_ID, JOB_DESCRIPTION,
 MIN_SALARY, MAX_SALARY, VACANCIES)
VALUES
(107, 'Data Analyst', 7, 'Data analysis and reporting',
 350000, 700000, 2);

COMMIT;




INSERT INTO ER_CANDIDATE VALUES
(1001, 'Diksha Rajwani', 'diksha.rajwani@example.com',
 '9000000001', 'MBA Human Resources', 1, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1002, 'Chahat Tekwani', 'chahat.tekwani@example.com',
 '9000000002', 'BSc Computer Science', 1, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1003, 'Karan Saini', 'karan.saini@example.com',
 '9000000003', 'BTech Computer Engineering', 2, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1004, 'Shravani Salunkhe', 'shravani.salunkhe@example.com',
 '9000000004', 'MSc Data Science', 1, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
```

```
(1005, 'Gauri Sardal', 'gauri.sardal@example.com',
 '9000000005', 'BCom Finance', 1, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1006, 'Yug Prajapati', 'yug.prajapati@example.com',
 '9000000006', 'BTech Information Technology', 2, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1007, 'Jatin Rawlani', 'jatin.rawlani@example.com',
 '9000000007', 'BSc Computer Science', 1, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1008, 'Soumya Ramani', 'soumya.ramani@example.com',
 '9000000008', 'MBA Finance', 2, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1009, 'Aarya Sanwre', 'aarya.sanwre@example.com',
 '9000000009', 'MSc Data Analytics', 1, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1010, 'Kshitij Sachan', 'kshitij.sachan@example.com',
 '9000000010', 'BTech Computer Engineering', 2, SYSDATE);

INSERT INTO ER_CANDIDATE VALUES
(1011, 'Sahil Varlani', 'sahil.varlani@example.com',
 '9000000011', 'BSc Information Technology', 1, SYSDATE);

COMMIT;
```

```
INSERT INTO ER_APPLICATION VALUES
(5001, 1001, 101, SYSDATE, 'SHORTLISTED',
 'Shortlisted for HR recruitment');

INSERT INTO ER_APPLICATION VALUES
(5002, 1002, 102, SYSDATE, 'APPLIED',
 'Application received');

INSERT INTO ER_APPLICATION VALUES
(5003, 1003, 102, SYSDATE, 'INTERVIEW',
 'Technical interview in progress');

INSERT INTO ER_APPLICATION VALUES
(5004, 1004, 104, SYSDATE, 'SHORTLISTED',
 'Shortlisted for data science');

INSERT INTO ER_APPLICATION VALUES
(5005, 1005, 103, SYSDATE, 'SELECTED',
 'Selected for financial analyst role');

INSERT INTO ER_APPLICATION VALUES
(5006, 1006, 102, SYSDATE, 'HIRED',
 'Recruitment completed');
```

```
INSERT INTO ER_APPLICATION VALUES
(5007, 1007, 105, SYSDATE, 'REJECTED',
 'Application not selected');

INSERT INTO ER_APPLICATION VALUES
(5008, 1008, 103, SYSDATE, 'INTERVIEW',
 'Finance interview scheduled');

INSERT INTO ER_APPLICATION VALUES
(5009, 1009, 107, SYSDATE, 'SHORTLISTED',
 'Shortlisted for data analytics');

INSERT INTO ER_APPLICATION VALUES
(5010, 1010, 105, SYSDATE, 'APPLIED',
 'Application under review');

INSERT INTO ER_APPLICATION VALUES
(5011, 1011, 106, SYSDATE, 'APPLIED',
 'Application received');

COMMIT;




SELECT
   C.CANDIDATE_ID,
   C.FULL_NAME,
```

```
    D.DEPT_NAME,
    J.JOB_TITLE,
    A.APPLICATION_STATUS
FROM ER_CANDIDATE C
JOIN ER_APPLICATION A
    ON C.CANDIDATE_ID = A.CANDIDATE_ID
JOIN ER_JOB J
    ON A.JOB_ID = J.JOB_ID
JOIN ER_DEPARTMENT D
    ON J.DEPT_ID = D.DEPT_ID
ORDER BY C.CANDIDATE_ID;
```

```
SELECT APPLICATION_STATUS, COUNT(*) AS TOTAL
FROM ER_APPLICATION
GROUP BY APPLICATION_STATUS
ORDER BY APPLICATION_STATUS;
```

```
SELECT TABLE_NAME
FROM USER_TABLES
WHERE TABLE_NAME LIKE 'ER_%'
ORDER BY TABLE_NAME;
```

```
CREATE OR REPLACE VIEW ER_V_CANDIDATE_APPLICATION AS
SELECT
   C.CANDIDATE_ID,
   C.FULL_NAME,
   C.EMAIL,
   C.PHONE,
   C.QUALIFICATION,
   C.EXPERIENCE_YEARS,
   A.APPLICATION_ID,
   A.APPLICATION_DATE,
   A.APPLICATION_STATUS,
   J.JOB_ID,
   J.JOB_TITLE,
   D.DEPT_NAME
FROM ER_CANDIDATE C
JOIN ER_APPLICATION A
   ON C.CANDIDATE_ID = A.CANDIDATE_ID
JOIN ER_JOB J
   ON A.JOB_ID = J.JOB_ID
JOIN ER_DEPARTMENT D
   ON J.DEPT_ID = D.DEPT_ID;
```

```
SELECT * FROM ER_V_CANDIDATE_APPLICATION;
```

```
UPDATE ER_CANDIDATE
SET FULL_NAME = 'Karan Anil Saini'
WHERE CANDIDATE_ID = 1003;

COMMIT;
```

```
SELECT CANDIDATE_ID, FULL_NAME, JOB_TITLE
FROM ER_V_CANDIDATE_APPLICATION
WHERE CANDIDATE_ID = 1003;
```

```
CREATE OR REPLACE VIEW ER_V_INTERVIEW_DETAILS AS
SELECT
  C.CANDIDATE_ID,
  C.FULL_NAME,
  C.EMAIL,
  A.APPLICATION_ID,
  J.JOB_TITLE,
```

```
    I.INTERVIEW_ID,
    I.INTERVIEW_ROUND,
    I.INTERVIEW_DATE,
    I.INTERVIEW_MODE,
    I.INTERVIEWER_NAME,
    I.SCORE,
    I.FEEDBACK,
    I.INTERVIEW_STATUS
FROM ER_CANDIDATE C
JOIN ER_APPLICATION A
    ON C.CANDIDATE_ID = A.CANDIDATE_ID
JOIN ER_JOB J
    ON A.JOB_ID = J.JOB_ID
JOIN ER_INTERVIEW I
    ON A.APPLICATION_ID = I.APPLICATION_ID;




SELECT * FROM ER_V_INTERVIEW_DETAILS;




CREATE TABLE ER_APPLICATION_AUDIT (
    AUDIT_ID NUMBER PRIMARY KEY,
    APPLICATION_ID NUMBER NOT NULL,
    OLD_STATUS VARCHAR2(25),
```

```
  NEW_STATUS VARCHAR2(25),
  CHANGE_DATE TIMESTAMP DEFAULT SYSTIMESTAMP,
  CHANGED_BY VARCHAR2(128)
);



CREATE SEQUENCE ER_AUDIT_SEQ
START WITH 1
INCREMENT BY 1;



CREATE OR REPLACE TRIGGER ER_APP_STATUS_AUDIT
AFTER UPDATE OF APPLICATION_STATUS
ON ER_APPLICATION
FOR EACH ROW
WHEN (OLD.APPLICATION_STATUS <> NEW.APPLICATION_STATUS)
BEGIN
  INSERT INTO ER_APPLICATION_AUDIT (
    AUDIT_ID,
    APPLICATION_ID,
    OLD_STATUS,
    NEW_STATUS,
    CHANGE_DATE,
    CHANGED_BY
```

```
  )
  VALUES (
    ER_AUDIT_SEQ.NEXTVAL,
    :OLD.APPLICATION_ID,
    :OLD.APPLICATION_STATUS,
    :NEW.APPLICATION_STATUS,
    SYSTIMESTAMP,
    USER
  );
END;
/
```

```
UPDATE ER_APPLICATION
SET APPLICATION_STATUS = 'SHORTLISTED'
WHERE APPLICATION_ID = 5002;

COMMIT;

SELECT *
FROM ER_APPLICATION_AUDIT
ORDER BY AUDIT_ID;
```

```
SELECT OBJECT_NAME, OBJECT_TYPE, STATUS
FROM USER_OBJECTS
WHERE OBJECT_NAME LIKE 'ER_V_%'
  OR OBJECT_NAME = 'ER_APP_STATUS_AUDIT'
ORDER BY OBJECT_TYPE, OBJECT_NAME;
```

```
CREATE SEQUENCE ER_CANDIDATE_SEQ
START WITH 1012
INCREMENT BY 1
NOCYCLE;

CREATE SEQUENCE ER_APPLICATION_SEQ
START WITH 5012
INCREMENT BY 1
NOCYCLE;

CREATE SEQUENCE ER_INTERVIEW_SEQ
START WITH 1
INCREMENT BY 1
NOCYCLE;

CREATE SEQUENCE ER_EMPLOYEE_SEQ
START WITH 1
INCREMENT BY 1
NOCYCLE;
```

```
SELECT SEQUENCE_NAME
FROM USER_SEQUENCES
WHERE SEQUENCE_NAME LIKE 'ER_%SEQ'
ORDER BY SEQUENCE_NAME;




CREATE OR REPLACE PROCEDURE ER_ADD_CANDIDATE (
   P_FULL_NAME IN VARCHAR2,
   P_EMAIL IN VARCHAR2,
   P_PHONE IN VARCHAR2,
   P_QUALIFICATION IN VARCHAR2,
   P_EXPERIENCE IN NUMBER
)
AS
BEGIN
   INSERT INTO ER_CANDIDATE (
      CANDIDATE_ID,
      FULL_NAME,
      EMAIL,
      PHONE,
      QUALIFICATION,
      EXPERIENCE_YEARS,
```

```
    CREATED_DATE
  )
  VALUES (
    ER_CANDIDATE_SEQ.NEXTVAL,
    P_FULL_NAME,
    P_EMAIL,
    P_PHONE,
    P_QUALIFICATION,
    P_EXPERIENCE,
    SYSDATE
  );

  DBMS_OUTPUT.PUT_LINE(
    'Candidate added successfully: ' || P_FULL_NAME
  );

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    RAISE_APPLICATION_ERROR(
      -20001,
      'Candidate email already exists.'
    );
END;
/
```

```
SET SERVEROUTPUT ON;




EXEC ER_ADD_CANDIDATE( 'Diksha Rajwani',   'diksha.new@example.com',
'9111111111',   'MBA Human Resources',   1);




SELECT *
FROM ER_CANDIDATE
ORDER BY CANDIDATE_ID;




CREATE OR REPLACE PROCEDURE ER_APPLY_FOR_JOB (
   P_CANDIDATE_ID IN NUMBER,
   P_JOB_ID IN NUMBER
)
AS
BEGIN
   INSERT INTO ER_APPLICATION (
      APPLICATION_ID,
      CANDIDATE_ID,
      JOB_ID,
```

```
    APPLICATION_DATE,
    APPLICATION_STATUS
  )
  VALUES (
    ER_APPLICATION_SEQ.NEXTVAL,
    P_CANDIDATE_ID,
    P_JOB_ID,
    SYSDATE,
    'APPLIED'
  );

  DBMS_OUTPUT.PUT_LINE(
    'Application submitted successfully.'
  );

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    RAISE_APPLICATION_ERROR(
      -20002,
      'This candidate has already applied for this job, or an application ID conflicts.'
    );
  WHEN OTHERS THEN
    IF SQLCODE = -2291 THEN
      RAISE_APPLICATION_ERROR(
        -20003,
        'Candidate or job does not exist.'
      );
    ELSE
```

```
        RAISE;
    END IF;
END;
/




EXEC ER_APPLY_FOR_JOB(1012, 101);

COMMIT;




SELECT *
FROM ER_APPLICATION
ORDER BY APPLICATION_ID;




CREATE OR REPLACE PROCEDURE ER_UPDATE_STATUS (
  P_APPLICATION_ID IN NUMBER,
  P_NEW_STATUS IN VARCHAR2
)
AS
BEGIN
```

```
  UPDATE ER_APPLICATION
  SET APPLICATION_STATUS = UPPER(P_NEW_STATUS)
  WHERE APPLICATION_ID = P_APPLICATION_ID;

  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(
      -20004,
      'Application ID not found.'
    );
  END IF;

  DBMS_OUTPUT.PUT_LINE(
    'Application status updated successfully.'
  );
END;
/




EXEC ER_UPDATE_STATUS(5012, 'SHORTLISTED');

COMMIT;




SELECT APPLICATION_ID, APPLICATION_STATUS
```

```
FROM ER_APPLICATION
WHERE APPLICATION_ID = 5012;

SELECT *
FROM ER_APPLICATION_AUDIT
ORDER BY AUDIT_ID;




SELECT OBJECT_NAME, STATUS
FROM USER_OBJECTS
WHERE OBJECT_NAME IN (
   'ER_ADD_CANDIDATE',
   'ER_APPLY_FOR_JOB',
   'ER_UPDATE_STATUS'
);




SHOW ERRORS PROCEDURE ER_ADD_CANDIDATE;
SHOW ERRORS PROCEDURE ER_APPLY_FOR_JOB;
SHOW ERRORS PROCEDURE ER_UPDATE_STATUS;
```

```
CREATE OR REPLACE PROCEDURE ER_SCHEDULE_INTERVIEW
(
  P_APPLICATION_ID IN NUMBER,
  P_INTERVIEW_ROUND IN VARCHAR2,
  P_INTERVIEW_DATE IN DATE,
  P_INTERVIEW_MODE IN VARCHAR2,
  P_INTERVIEWER_NAME IN VARCHAR2
)
AS
BEGIN
  INSERT INTO ER_INTERVIEW
  (
    INTERVIEW_ID,
    APPLICATION_ID,
    INTERVIEW_ROUND,
    INTERVIEW_DATE,
    INTERVIEW_MODE,
    INTERVIEWER_NAME,
    SCORE,
    FEEDBACK,
    INTERVIEW_STATUS
  )
  VALUES
  (
    ER_INTERVIEW_SEQ.NEXTVAL,
    P_APPLICATION_ID,
    P_INTERVIEW_ROUND,
```

```
    P_INTERVIEW_DATE,
    UPPER(P_INTERVIEW_MODE),
    P_INTERVIEWER_NAME,
    NULL,
    NULL,
    'SCHEDULED'
  );

  DBMS_OUTPUT.PUT_LINE('Interview scheduled successfully.');

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    RAISE_APPLICATION_ERROR(-20010, 'Duplicate interview record.');

  WHEN OTHERS THEN
    RAISE;
END;
/




SET SERVEROUTPUT ON;
```

```
EXEC ER_SCHEDULE_INTERVIEW(5012, 'Technical Round', TO_DATE('25-10-2026','DD-MM-YYYY'), 'Online', 'Rahul Mehta');




COMMIT;




SELECT *
FROM ER_INTERVIEW
ORDER BY INTERVIEW_ID;




CREATE OR REPLACE PROCEDURE ER_APPLICATION_REPORT
AS
  CURSOR C_APPLICATIONS IS
    SELECT
      A.APPLICATION_ID,
      C.FULL_NAME,
      J.JOB_TITLE,
      D.DEPT_NAME,
      A.APPLICATION_STATUS,
```

```
      A.APPLICATION_DATE
    FROM ER_APPLICATION A
    JOIN ER_CANDIDATE C
      ON A.CANDIDATE_ID = C.CANDIDATE_ID
    JOIN ER_JOB J
      ON A.JOB_ID = J.JOB_ID
    JOIN ER_DEPARTMENT D
      ON J.DEPT_ID = D.DEPT_ID
    ORDER BY A.APPLICATION_ID;

  V_APP_ID ER_APPLICATION.APPLICATION_ID%TYPE;
  V_NAME ER_CANDIDATE.FULL_NAME%TYPE;
  V_JOB ER_JOB.JOB_TITLE%TYPE;
  V_DEPT ER_DEPARTMENT.DEPT_NAME%TYPE;
  V_STATUS ER_APPLICATION.APPLICATION_STATUS%TYPE;
  V_DATE ER_APPLICATION.APPLICATION_DATE%TYPE;

BEGIN
  OPEN C_APPLICATIONS;

  LOOP
    FETCH C_APPLICATIONS
    INTO V_APP_ID, V_NAME, V_JOB, V_DEPT, V_STATUS, V_DATE;

    EXIT WHEN C_APPLICATIONS%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      'Application ID: ' || V_APP_ID ||
```

```
        ' | Candidate: ' || V_NAME ||
        ' | Job: ' || V_JOB ||
        ' | Department: ' || V_DEPT ||
        ' | Status: ' || V_STATUS ||
        ' | Date: ' || TO_CHAR(V_DATE, 'DD-MON-YYYY')
      );
  END LOOP;

  CLOSE C_APPLICATIONS;
END;
/




SET SERVEROUTPUT ON;
EXEC ER_APPLICATION_REPORT;




DESC ER_INTERVIEW;




CREATE OR REPLACE PROCEDURE ER_SCHEDULE_INTERVIEW
```

```
(
   P_APPLICATION_ID IN NUMBER,
   P_INTERVIEW_ROUND IN NUMBER,
   P_INTERVIEW_DATE IN DATE,
   P_INTERVIEW_MODE IN VARCHAR2,
   P_INTERVIEWER_NAME IN VARCHAR2
)
AS
BEGIN
   INSERT INTO ER_INTERVIEW
   (
      INTERVIEW_ID,
      APPLICATION_ID,
      INTERVIEW_ROUND,
      INTERVIEW_DATE,
      INTERVIEW_MODE,
      INTERVIEWER_NAME,
      SCORE,
      FEEDBACK,
      INTERVIEW_STATUS
   )
   VALUES
   (
      ER_INTERVIEW_SEQ.NEXTVAL,
      P_APPLICATION_ID,
      P_INTERVIEW_ROUND,
      P_INTERVIEW_DATE,
      UPPER(P_INTERVIEW_MODE),
```

```
    P_INTERVIEWER_NAME,
    NULL,
    NULL,
    'SCHEDULED'
  );

  DBMS_OUTPUT.PUT_LINE('Interview scheduled successfully.');

EXCEPTION
  WHEN OTHERS THEN
    RAISE;
END;
/
```

```
EXEC ER_SCHEDULE_INTERVIEW(5012, 1, TO_DATE('25-10-2026','DD-MM-YYYY'),
'Online', 'Rahul Mehta');
```

```
COMMIT;

SELECT *
FROM ER_INTERVIEW
```

```
ORDER BY INTERVIEW_ID;
```

```
CREATE OR REPLACE PROCEDURE ER_APPLICATION_REPORT
AS
  CURSOR C_APPLICATIONS IS
    SELECT
      A.APPLICATION_ID,
      C.FULL_NAME,
      J.JOB_TITLE,
      D.DEPT_NAME,
      A.APPLICATION_STATUS,
      A.APPLICATION_DATE
    FROM ER_APPLICATION A
    JOIN ER_CANDIDATE C
      ON A.CANDIDATE_ID = C.CANDIDATE_ID
    JOIN ER_JOB J
      ON A.JOB_ID = J.JOB_ID
    JOIN ER_DEPARTMENT D
      ON J.DEPT_ID = D.DEPT_ID
    ORDER BY A.APPLICATION_ID;

  V_APP_ID ER_APPLICATION.APPLICATION_ID%TYPE;
  V_NAME ER_CANDIDATE.FULL_NAME%TYPE;
```

```
  V_JOB ER_JOB.JOB_TITLE%TYPE;
  V_DEPT ER_DEPARTMENT.DEPT_NAME%TYPE;
  V_STATUS ER_APPLICATION.APPLICATION_STATUS%TYPE;
  V_DATE ER_APPLICATION.APPLICATION_DATE%TYPE;

BEGIN
  OPEN C_APPLICATIONS;

  LOOP
    FETCH C_APPLICATIONS
    INTO V_APP_ID, V_NAME, V_JOB, V_DEPT, V_STATUS, V_DATE;

    EXIT WHEN C_APPLICATIONS%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(
      'Application: ' || V_APP_ID ||
      ' | Candidate: ' || V_NAME ||
      ' | Job: ' || V_JOB ||
      ' | Department: ' || V_DEPT ||
      ' | Status: ' || V_STATUS ||
      ' | Date: ' || TO_CHAR(V_DATE, 'DD-MON-YYYY')
    );
  END LOOP;

  CLOSE C_APPLICATIONS;
END;
/
```

```
SET SERVEROUTPUT ON;
EXEC ER_APPLICATION_REPORT;
```

```
CREATE OR REPLACE PROCEDURE ER_HIRE_CANDIDATE
(
   P_APPLICATION_ID IN NUMBER,
   P_DESIGNATION IN VARCHAR2,
   P_ANNUAL_SALARY IN NUMBER
)
AS
   V_CANDIDATE_ID NUMBER;
   V_STATUS ER_APPLICATION.APPLICATION_STATUS%TYPE;
   V_EMPLOYEE_COUNT NUMBER;
BEGIN
   SELECT CANDIDATE_ID, APPLICATION_STATUS
   INTO V_CANDIDATE_ID, V_STATUS
   FROM ER_APPLICATION
   WHERE APPLICATION_ID = P_APPLICATION_ID
   FOR UPDATE;

   IF V_STATUS = 'HIRED' THEN
```

```
    RAISE_APPLICATION_ERROR(
      -20020,
      'This application is already marked as hired.'
    );
END IF;

SELECT COUNT(*)
INTO V_EMPLOYEE_COUNT
FROM ER_EMPLOYEE
WHERE APPLICATION_ID = P_APPLICATION_ID;

IF V_EMPLOYEE_COUNT > 0 THEN
    RAISE_APPLICATION_ERROR(
      -20021,
      'An employee record already exists for this application.'
    );
END IF;

UPDATE ER_APPLICATION
SET APPLICATION_STATUS = 'HIRED'
WHERE APPLICATION_ID = P_APPLICATION_ID;

INSERT INTO ER_EMPLOYEE
(
    EMPLOYEE_ID,
    APPLICATION_ID,
    CANDIDATE_ID,
    JOINING_DATE,
```

```
      DESIGNATION,
      ANNUAL_SALARY,
      EMPLOYMENT_STATUS
    )
    VALUES
    (
      ER_EMPLOYEE_SEQ.NEXTVAL,
      P_APPLICATION_ID,
      V_CANDIDATE_ID,
      SYSDATE,
      P_DESIGNATION,
      P_ANNUAL_SALARY,
      'ACTIVE'
    );

    DBMS_OUTPUT.PUT_LINE(
      'Candidate hired successfully. Candidate ID: ' ||
      V_CANDIDATE_ID
    );

EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(
        -20022,
        'Application ID does not exist.'
      );
END;
/
```

```
SELECT APPLICATION_ID, CANDIDATE_ID, APPLICATION_STATUS
FROM ER_APPLICATION
ORDER BY APPLICATION_ID;
```

```
EXEC ER_HIRE_CANDIDATE(5012, 'Software Developer', 600000);
```

```
SELECT *
FROM ER_EMPLOYEE
ORDER BY EMPLOYEE_ID;
```

```
COMMIT;
```

```
SELECT OBJECT_NAME, STATUS
FROM USER_OBJECTS
WHERE OBJECT_NAME IN (
   'ER_APPLICATION_REPORT',
   'ER_HIRE_CANDIDATE'
);
```

```
SET SERVEROUTPUT ON;
EXEC ER_APPLICATION_REPORT;
```

```
SELECT APPLICATION_STATUS, COUNT(*) AS TOTAL
FROM ER_APPLICATION
GROUP BY APPLICATION_STATUS
ORDER BY APPLICATION_STATUS;
```

```
SELECT
   D.DEPT_NAME,
   COUNT(J.JOB_ID) AS TOTAL_JOBS,
```

```
    NVL(SUM(J.VACANCIES), 0) AS TOTAL_VACANCIES
FROM ER_DEPARTMENT D
LEFT JOIN ER_JOB J
    ON D.DEPT_ID = J.DEPT_ID
    AND J.JOB_STATUS = 'OPEN'
GROUP BY D.DEPT_ID, D.DEPT_NAME
ORDER BY D.DEPT_NAME;
```

```
SELECT
    C.CANDIDATE_ID,
    C.FULL_NAME,
    J.JOB_TITLE,
    D.DEPT_NAME,
    A.APPLICATION_STATUS
FROM ER_CANDIDATE C
JOIN ER_APPLICATION A
    ON C.CANDIDATE_ID = A.CANDIDATE_ID
JOIN ER_JOB J
    ON A.JOB_ID = J.JOB_ID
JOIN ER_DEPARTMENT D
    ON J.DEPT_ID = D.DEPT_ID
ORDER BY C.FULL_NAME;
```

```
COMMIT;
EXIT;
```

```
SELECT TABLE_NAME
FROM USER_TABLES
WHERE TABLE_NAME LIKE 'ER_%'
ORDER BY TABLE_NAME;
```

```
SELECT OBJECT_NAME, OBJECT_TYPE, STATUS
FROM USER_OBJECTS
WHERE OBJECT_NAME LIKE 'ER_%'
ORDER BY OBJECT_TYPE, OBJECT_NAME;
```

```
SELECT APPLICATION_ID, CANDIDATE_ID, JOB_ID, APPLICATION_STATUS
FROM ER_APPLICATION
```

```
ORDER BY APPLICATION_ID;
```

```
SET SERVEROUTPUT ON;
EXEC ER_APPLICATION_REPORT;
```